Character class with warrior and mage subclasses

In [ ]:
```python
class Character:

    """Represents a character in the RPG"""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10, heal_up:
        """
        Initialise the Character
        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100)
            damage (int): Damage dealt when character attacks (default = 10)
            heal_up (int): Default healing amount (default = 5)
        """
        self.name = name
        self._hp = hp_max  # Note: _hp indicates private attribute
        self.hp_max = hp_max
        self.damage = damage
        self.heal_up = heal_up
        self.weapon = weapon

    @property
    def is_alive(self) -> bool:
        """Check if the character is still alive"""
        return self._hp > 0

    def attack(self, target) -> None:
        """Attack and deal damage to target character"""
        if self.weapon is not None:
            weapon_damage = self.weapon.roll_damage()
            target.take_damage(weapon_damage)
        else:
            target.take_damage(self.damage)

    def take_damage(self, amount: int) -> None:
        """Character takes damage"""
        self._hp -= amount
        self._hp = max(self._hp, 0)

    def heal(self, amount: int = None) -> None:
        """Heal the character"""
        heal_amount = amount if amount is not None else self.heal_up
        self._hp += heal_amount
        self._hp = min(self._hp, self.hp_max)  # HP cannot exceed max HP

    def introduce(self) -> None:
        """Creates string that introduces character"""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of character object"""
        return f"<Character name={self.name} hp={self.hp} damage={self.damage}>"

    def __str__(self) -> str:
        """String representation of character object"""
        return f"Character: {self.name} with {self.hp} HP and {self.damage} dama
```

```
    @property
    def hp(self) -> int:
        """Public read-only property to inspect HP without allowing direct, unsa
        return self._hp



class Warrior(Character):
    def __init__(self, name, hp_max = 100, damage = 10, heal_up = 5, armor: int
        super().__init__(name, hp_max, damage, heal_up, weapon)
        self.armor = armor

    def take_damage(self, amount: int) -> None:
        reduced_damage = max(amount-self.armor, 1)
        super().take_damage(reduced_damage)


class Mage(Character):
    def __init__(self, name, hp_max = 100, damage = 10, heal_up = 5, weapon = No
        super().__init__(name, hp_max, damage, heal_up, weapon)
        self.mana = mana
        self.mana_max = mana_max

    def attack(self, target):
        if self.mana >= 10:
            self.mana -= 10
            spell_damage = self.damage*2
            target.take_damage(spell_damage)
        else:
            self.mana = min(self.mana += 5, mana_max)
            mana = max(mana, 30)
            super().attack(target)
```

Die class

In [ ]:

```
import random

class Dice:

    def __init__(self, sides: int = 6) -> None:
        """Initialise the dice
            Args:
            Number of sides
        """
        if sides < 2:
            raise ValueError('Die must have at least 2 sides')
        self.sides = sides


    def roll(self) -> int:
        """ Roll method for rolling dice"""
        dice_roll = random.randint(1, self.sides)
        return dice_roll

    def __repr__(self) -> str:
        return f"Dice(sides = {self.sides})"

    def __call__(self) -> int:
```

```
        """ Method allowing individual objects to be called like functions"""
        return self.roll()
```

Weapon class

In [ ]:
```
class Weapon:
    def __init__(self, name: str, damage_dice: Dice):
        """Intialising the weapon class
        2 attributes: name and damage_dice"""

        self.name = name
        self.damage_dice = damage_dice


    def roll_damage(self):
        return self.damage_dice.roll()
```

Battle class

In [ ]:
```
class Battle():
    """Create a class for battles between heros and enemies"""
    def __init__(self, hero: Character, enemy: Character):
        self.hero = hero
        self.enemy = enemy

    def play_round(self) -> None:
        """Method that implements combat logic"""
        self.hero.attack(self.enemy)
        if self.enemy.is_alive:
            self.enemy.attack(self.hero)


    def simulate_battle(self) -> None:
        """Method that simulates a complete battle"""
        while self.hero.is_alive and self.enemy.is_alive:
            self.play_round()
```